

程式語言

Programming Languages — Class 2

政大資碩 115-1 陳家宏

選課/加選

1. 請儘量使用學校的系統
2. 使用系統無法加選，麻煩到樓上找佳慧助教

今天三堂課

時段	內容
第 1 堂	精讀 Ch1.2 : free / bound 、 subst 、 follow the grammar 的極限
第 2 堂	Ch1.3 : 輔助程序與 context argument
第 3 堂	HW1 發布 + 課堂習題演練

課文裡的三句 slogan

slogan	出處	意義
Proof by Structural Induction	p.12	$\text{IH}(\text{simple}):\text{true}, \text{IH}(\text{sub}(s)):\text{true} \rightarrow \text{IH}(s):\text{true} \Rightarrow \text{IH}(\text{all}):\text{true}$
Smaller-Subproblem Principle	p.12	若一個問題可化約成更小的、同形式的子問題，就可以用遞迴來解它
Follow the Grammar!	p.22	資料的形狀決定程式的形狀

今天要加上第四句，並且要看到第三句失效。

開場：30 秒小測

```
(lambda (x)
  (lambda (y)
    (x y z)))
```

- Q1: `x`、`y`、`z` 三個變數，哪些是 **free**？
- Q2: 可以用 Ch 1.2 的 `occurs-free?` 來檢查嗎？

第 1 堂

Ch1.2 精讀

回到 LcExp 文法

```
LcExp ::= Identifier  
      ::= (lambda (Identifier) LcExp)  
      ::= (LcExp LcExp)
```

接下來八張投影片，只依賴這三行。

occurs free 的定義 (口語)

- 意義

這個符號 x 的意義必須由「外層環境」提供嗎？

- 課文定義 (口語)

變數 x 在 e 中自由出現 (occurs free)，是指 x 在 e 裡有某次出現，而那次出現不在任何綁定 x 的 lambda 之內。

課文這句可讀，但不可運算。

把它換成三條可以逐條檢查的 rules。

occurs free 的定義 (規則)

1. 若 e 是一個變數 (即 Identifier) , 則 x 在 e 中 free 若且唯若 x 就是 e
 2. 若 e 是 $(\text{lambda } (y) e')$, 則 x 在 e 中 free 若且唯若
 y 不是 x , 且 x 在 e' 中 free
 3. 若 e 是 $(e1\ e2)$, 則 x 在 e 中 free 若且唯若
 x 在 $e1$ 或(註) $e2$ 中 free
- 註 : inclusive OR

這就是 follow the grammar——rules 都跟著文法走。

兩種定義等價嗎？

- 課文

你應該自行驗證到確信為止：這組規則確實表達了「出現的位置不在任何綁定 x 的 λ 之內」這個說法。

You should **convince yourself** that these rules capture the notion of occurring “not inside a λ -binding of x .”

- 比較

	內容	特色
前一頁	口語	一個非遞迴的量化敘述
這一頁	機械可檢查的 rules	對 e 的結構做遞迴（ x 不變）

注意：convince yourself 在數學寫作裡是一個標記，意思是「作者判斷這件事為真，不給證明，這是你的功課。」

occurs-free?

```
;; occurs-free? : Sym * LcExp -> Bool
;; usage: returns #t if var occurs free in exp
(define occurs-free?
  (lambda (var exp)
    (cond
      ((symbol? exp) (eqv? var exp))
      ((eqv? (car exp) 'lambda)
       (and (not (eqv? var (car (cadr exp))))
            (occurs-free? var (caddr exp))))
      (else
       (or (occurs-free? var (car exp))
           (occurs-free? var (cadr exp)))))))
```

三個 `cond` 分支 $\leftrightarrow$ 三條 production $\leftrightarrow$ 三條 rules。

Q2 的答案：可以跑

```
(occurs-free? 'z '(lambda (x) (lambda (y) (x y z))))  
; => #f
```

沒有報錯。沒有警告。回傳 `#f`。

但 Q1 的正確答案是：`z` 由外層提供意義，它是 **free**，應該是 `#t`。

程式沒有壞掉。它安靜地給了一個錯的答案。

為什麼會是 #f

```
第 2 條: z ≠ x, 往 (lambda (y) (x y z)) 裡走  
第 2 條: z ≠ y, 往 (x y z) 裡走  
第 3 條: (or (occurs-free? 'z 'x)      ; ← (car exp)  
            (occurs-free? 'z 'y))    ; ← (cadr exp)  
=> #f
```

(x y z) 有三個元素。

第 3 條 rule 只認得 e1 和 e2，程式也只取 car 和 cadr。

z 從頭到尾沒有被看到。

(x y z) 不是 LcExp——文法只允許 (LcExp LcExp)。

違反 contract 會發生什麼

```
;; occurs-free? : Sym * LcExp -> Bool
```

contract 說輸入必須是 `LcExp`。我們餵了不是 `LcExp` 的東西進去。

你可能期待	實際發生
報錯	沒有
回傳正確答案	沒有
—	回傳一個看起來很正常的 <code>#f</code>

前提被違反時，程式沒有義務做任何事——包括沒有義務告訴你出事了。

先修好它：正確的寫法

`(x y z)` 在單參數的 lambda calculus 中要寫成 `((x y) z)`：

```
(occurs-free? 'z '(lambda (x) (lambda (y) ((x y) z))))  
; => #t  
(occurs-free? 'x '(lambda (x) (lambda (y) ((x y) z))))  
; => #f  
(occurs-free? 'y '(lambda (x) (lambda (y) ((x y) z))))  
; => #f
```

答案：只有 **z** 是 **free**。現在 **#t** 了。

同一支程式、同一個問題，只改了輸入的形狀。
文法寫得精確，程式才有辦法精確。

第二個例子：S-list 上的 subst

課本 Definition 1.1.6 的寫法：

```
S-list ::= ({S-exp}*)  
S-exp  ::= Symbol | S-list
```

Kleene star 描述集合很簡潔，但對寫程式沒有幫助——

`{S-exp}*` 沒有告訴你要怎麼把一個 s-list 拆開。

第一步：把 Kleene star 消掉。

改寫成可以遞迴的形式

```
S-list ::= ()  
        ::= (S-exp . S-list)  
S-exp  ::= Symbol | S-list
```

同一個集合，換一種寫法。差別在於：

寫法	告訴你什麼
<code>{S-exp}*</code>	這個集合有哪些元素
<code>() (S-exp . S-list)</code>	一個元素可以怎麼拆

第二種寫法直接指出要對 s-list 的 `car` 和 `cdr` 做遞迴。

文法不只描述資料，它還要能指導程式。

描述同一個集合的兩種文法，對寫程式的用處可以差很多。

兩個 nonterminal $\Rightarrow$ 兩個函式

```
S-list ::= () | (S-exp . S-list)
S-exp  ::= Symbol | S-list
```

這是第一個輸入文法含兩個 nonterminal 的例子。

```
:: subst : Sym * Sym * S-list -> S-list
:: subst-in-s-exp : Sym * Sym * S-exp -> S-exp
```

兩個互相遞迴的 nonterminal $\Rightarrow$ 兩個互相遞迴的函式。

subst

```
;; subst : Sym * Sym * S-list -> S-list
(define subst
  (lambda (new old slist)
    (if (null? slist)
        '()
        (cons (subst-in-s-exp new old (car slist))
                (subst new old (cdr slist))))))
```

兩個分支，對應 S-list 的兩條 production：

production	分支
()	回傳 '()
(S-exp . S-list)	car 交給 subst-in-s-exp ， cdr 交給 subst

subst-in-s-exp

```
;; subst-in-s-exp : Sym * Sym * S-exp -> S-exp
(define subst-in-s-exp
  (lambda (new old sexp)
    (if (symbol? sexp)
        (if (eqv? sexp old) new sexp)
        (subst new old sexp))))
```

```
(subst 'a 'b '((b c) (b () d)))
; => ((a c) (a () d))
```

注意最後一行：(subst new old sexp) 傳的是 sexp 本身，沒有變小。

同一個值被交給另一個函式重看一次。

文法說 S-exp 可以整個就是一個 S-list，所以這裡走的是 nonterminal，不是資料的子結構。

(HW1 的 1.11 會問：那為什麼還是會停？)

為什麼不合併成一個函式

技術上可以（課本 Exercise 1.12 的 inlining 就在做這件事）。

但合併之後：

- 函式同時處理兩種 nonterminal，型別不再單一
- contract 寫不乾淨：輸入到底是 S-list 還是 S-exp？
- 要驗證正確性時，歸納假設變成兩個混在一起

| 文法有幾個 nonterminal，就寫幾個函式。

但書一：它到底保證了什麼

follow the grammar 的宣稱不是「你不會寫錯」。

它的宣稱是：

把錯誤侷限在單一支裡——可除錯性（debuggability）。

錯了還是會錯——但你知道要去哪一格找。

一定要有型別、要有 unit test，程式才可能除錯嗎？

還是說，光是架構上的設計就能給你 local reasoning？

現場實驗：讓 occurs-free? 爛掉

需求變更：lambda 要支援多個參數。

```
LcExp ::= Identifier  
      ::= (lambda ({Identifier}*) LcExp)      ; 改這裡  
      ::= (LcExp {LcExp}*)                    ; 順便改這裡
```

看看程式要動哪裡：

```
(and (not (eqv? var (car (cadr exp))))      ; ← (car (cadr exp)) 是什麼?  
      (occurs-free? var (caddr exp)))  
(or (occurs-free? var (car exp))           ; ← 只處理兩個元素  
     (occurs-free? var (cadr exp)))
```

問題出在哪

```
(car (cadr exp))
```

讀這行程式，你看得出來它指的是「這個 lambda 的宣告變數」嗎？

- `cadr` 取出參數列，`car` 取第一個——寫死了「只有一個參數」
- `caddr` 寫死了「body 在第三個位置」
- `(or ... (car exp) ... (cadr exp))` 寫死了「應用只有兩個元素」

這支程式知道文法，但程式碼本身沒有說出來。

課本自己承認這件事

課本在 §1.2.4 寫完這支程式的下一句就承認了：

這支程式不如它應有的好讀——很難看出 `(car (cadr exp))` 指的是一個變數的宣告，也很難看出 `(caddr exp)` 指的是它的 body。

Ch2 的版本長這樣（今天不解釋，只讓你認得）：

```
(cases lc-exp exp
  (var-exp (var) ...)
  (lambda-exp (bound-var body) ...) ; ← 名字說出了角色
  (app-exp (rator rand) ...))
```

同樣的需求變更，在這個版本只要改 datatype 宣告，編譯器會告訴你哪些分支需要跟著改。

這是 week 3、week 4 的主題。今天先記住這個 before。

第 1 堂回顧

1. `occurs-free?` 的三個分支 = 文法的三條 production = 三條 rules
2. follow the grammar 保證的是**可除錯性**：錯誤被侷限在單一支裡
3. `(car (cadr exp))` 的脆弱，就是 Ch2 存在的理由

第 2 堂

Ch1.3 輔助程序與 Context Argument

一個看起來很簡單的需求

```
(number-elements '(a b c))  
; => ((0 a) (1 b) (2 c))
```

規格：把 `(v0 v1 v2 ...)` 變成 `((0 v0) (1 v1) (2 v2) ...)`。

用上一堂的配方試試看。文法是熟悉的：

```
List ::= () | (SchemeVal . List)
```

照配方寫，然後卡住

```
(define number-elements
  (lambda (lst)
    (if (null? lst)
        '()
        (cons ???
              (number-elements (cdr lst))))))
```

假設 `(number-elements '(b c))` 已經給我 `((0 b) (1 c))`。

我要的答案是 `((0 a) (1 b) (2 c))`。

每一個編號都要 +1。遞迴呼叫的結果不能直接用——要整個重走一遍。

問題的本質

follow the grammar 假設：

答案可以由「子結構的答案」直接組出來。

這裡不行。因為每個元素的編號，取決於它在**原始 list** 裡的位置——而子結構根本不知道自己被切掉了幾個元素。

缺的資訊不在資料裡，在脈絡裡。

開場預告的「第三句失效」，就是這裡。

解法：把問題變難一點

這聽起來很反直覺，但它是這一節的核心技巧：

| 解不出來，就把問題「推廣」(generalize)。

原問題：從 0 開始編號。

新問題：從 n 開始編號， n 是參數。

新問題更一般、看起來更難——但它可以直接遞迴。

number-elements-from

```
;; number-elements-from :  
;;   Listof(SchemeVal) * Int -> Listof(List(Int, SchemeVal))  
;; usage: (number-elements-from '(v0 v1 v2 ...) n)  
;;        = ((n v0) (n+1 v1) (n+2 v2) ...)  
(define number-elements-from  
  (lambda (lst n)  
    (if (null? lst)  
        '()  
        (cons (list n (car lst))  
                (number-elements-from (cdr lst) (+ n 1))))))  
  
(define number-elements  
  (lambda (lst)  
    (number-elements-from lst 0)))
```

觀察一：第四句 slogan

No Mysterious Auxiliaries! (課本 p.23)

定義輔助程序時，要說明它對「所有」引數的行為，
不只是最初呼叫的那組值。

為什麼這條重要？

- `number-elements` 只呼叫 `number-elements-from` 一次，用 `n = 0`
- 但遞迴過程中，`n` 會是 1、2、3.....
- 如果你只說明 `n = 0` 的情況，你就無法理解這支程式

反過來說：寫不出「對所有 `n` 都成立」的 usage 註解 $\Rightarrow$ 你還沒想清楚。

觀察二：兩個參數的角色完全不同

參數	遞迴時的變化	角色
lst	變小 (cdr)	被分解的資料，保證終止
n	變大 (+1)	脈絡的抽象

第二種參數，課本叫它 **context argument**，也叫 **inherited attribute**。

注意「變大」這件事。看到參數在遞迴時變大，直覺會警鈴大作——但就終止性而言，那是由第一個參數保證的，第二個怎麼變都沒關係。

第二個例子：vector 的和

先看 list 的版本（完全照配方）：

```
;; list-sum : Listof(Int) -> Int
(define list-sum
  (lambda (loi)
    (if (null? loi)
        0
        (+ (car loi)
            (list-sum (cdr loi)))))))
```

換成 vector，這招整個垮掉。

為什麼 vector 不能 follow the grammar

list 是歸納定義的：

```
List-of-Int ::= () | (Int . List-of-Int)
```

`cdr` 一個 list，還是一個 list——結構自己會分解。

vector 不是。 `#(1 2 3)` 沒有「尾巴也是一個 vector」這回事。
你只能用 index 去存取。

沒有歸納結構 $\Rightarrow$ 沒有文法可以 follow。

一樣的解法：推廣

原問題：求 $\sum_{i=0}^{\text{length}(v)-1} v_i$

推廣：把上界變成參數，求 $\sum_{i=0}^n v_i$ ，其中 $0 \leq n < \text{length}(v)$

現在遞迴的對象是整數 n ，而整數有歸納結構（ 0 和 $n+1$ ）。

找不到可以遞迴的結構，就自己造一個。

partial-vector-sum

```
;; partial-vector-sum : Vectorof(Int) * Int -> Int
;; usage: if 0 <= n < length(v), then
;;        (partial-vector-sum v n) = sum of v[0] .. v[n]
(define partial-vector-sum
  (lambda (v n)
    (if (zero? n)
        (vector-ref v 0)
        (+ (vector-ref v n)
            (partial-vector-sum v (- n 1)))))))
```

注意 usage 裡的 **if 0 <= n < length(v)** ——這是前提條件。
少了它，這支程式的規格是錯的。

vector-sum

```
;; vector-sum : Vectorof(Int) -> Int
(define vector-sum
  (lambda (v)
    (let ((n (vector-length v)))
      (if (zero? n)
          0
          (partial-vector-sum v (- n 1)))))))
```

`partial-vector-sum` 的規格是 $0 \leq n < \text{length}(v)$ 。

當 $\text{length}(v) = 0$ ，這個條件變成 $0 \leq n < 0$ ——沒有任何 n 滿足它。

不是「我們傳了一個壞的 n 」，是根本不存在好的 n 。

課本的說法：長度為 0 時，`partial-vector-sum` 不適用。

再看一次它的基底：`(vector-ref v 0)`。

它至少會加一個元素，表達不出「零個元素的和」。那個 `0` 只能由呼叫端給。

補充：同一個 nil，兩種錯誤訊息

```
(def users [{:name "Amy" :email "amy@nccu.edu.tw"}  
            {:name "Ben"} ; 忘了填 email  
            {:name "Cat" :email "cat@gmail.com"}])  
(defn domain-of [user] (second (str/split (:email user) #"@")))  
(frequencies (map domain-of users)) ; 統計各網域的人數
```

沒有 **spec**：錯誤指向 Java regex 的內部，看不出是哪位使用者出了問題

```
Execution error (NullPointerException) at java.util.regex.Matcher/getTextLength  
Cannot invoke "java.lang.CharSequence.length()" because "this.text" is null
```

加上 **spec**：指出是哪個函式、哪一筆資料、缺了什麼

```
;; (s/def ::email string?)  
;; (s/fdef domain-of :args (s/cat :user (s/keys :req-un [::email])))  
Execution error - invalid arguments to user/domain-of  
{:name "Ben"} - failed: (contains? % :email) at: [:user]
```

要讀懂一個 exception，需要足夠的 **context**：哪個函式、哪一筆資料、預期什麼。
把這些 context 放進 exception，debug 就會容易得多。

補充：在 Clojure 裡，vector 拆得開

Racket 的 vector 不能 `car`：

```
(car #(1 2 3)) ; error – vector 不是 pair
```

Clojure 的可以：

```
(first [1 2 3]) ;=> 1  
(rest  [1 2 3]) ;=> (2 3)
```

那剛才說的「vector 沒有歸納結構」是錯的嗎？

seq : Clojure 多的那一層視圖

```
(class (rest [1 2 3]))  
=> clojure.lang.PersistentVector$ChunkedSeq  
  
(vector? (rest [1 2 3]))  
=> false
```

← 拆完就不是 vector 了

first / rest 不直接作用在 vector 上。Clojure 先用 seq 把 collection 轉成一個序列視圖 (sequence interface)，那個視圖才是歸納定義的：

```
Seq ::= nil | (x . Seq)
```

誰提供歸納結構

	Racket vector	Clojure vector
資料本身	非歸納，只能 index	非歸納，只能 nth
歸納視圖	沒有	有： seq
拆解的結果	—	是 seq，不是 vector

EOPL 的論點沒有被推翻：要遞迴，就必須有歸納結構。

Clojure 把那個結構**做成另一層**，資料本身不必具備。

代價是拆出來的東西換了型別——你拿回 seq，不是 vector。

這也是為什麼 Clojure 算 vector 的和會用 reduce 而不是 first / rest：

reduce 直接走 vector 的內部結構，不必先蓋一層 seq。

第 2 堂回顧

1. `number-elements` 是 follow the grammar 的**第一個反例**（第三句失效）
2. 解不出來 $\Rightarrow$ **推廣**問題，多加一個參數
3. **No Mysterious Auxiliaries**：輔助程序要對所有引數有規格
4. **context argument**：遞迴時會變大的參數，終止性由另一個參數保證
5. `vector` 沒有歸納結構 $\Rightarrow$ 對 `index` 遞迴

第 3 堂

HW1 + 課堂演練

HW1：七個題號

題號	內容	難度
1.1 (只做 1、2 小題)	三種寫法定義集合	★
1.5	證明 LcExp 左右括號數相同	★★
1.11	為什麼 <code>subst-in-s-exp</code> 的遞迴會停	★
1.18	<code>swapper</code>	★
1.27	<code>flatten</code>	★★
1.31 + 1.33	bintree 介面 + <code>mark-leaves-with-red-depth</code>	★ / ★★

繳交期限：week 4 上課前。

每題學到什麼

題號	如果你答得出來，代表你懂了
1.1	同一個集合，三種寫法都寫得出來
1.5	結構歸納證明的完整流程
1.11	「為什麼會停」——答不出來代表整章沒懂
1.18	互遞迴的 nonterminal $\Rightarrow$ 互遞迴的函式
1.27	輸出結構與輸入結構 不同構 的第一題
1.31	constructor / predicate / extractor 的分離（Ch2 的橋）
1.33	context argument 的直接應用

先讀 §1.4 的開場白 (p.25–26)

課本在習題開始前，用一段話定好了所有題目共用的變數命名慣例。
不讀這一段，很多題目會看不懂輸入是什麼。

- `s` symbol
- `n` 非負整數（不是任意整數）
- `lst` list（任意元素）
- `loi` list of integers
- `los` list of symbols
- `slist` s-list
- `x` 任意 Scheme 值
- `pred` predicate：吃任意 Scheme 值，一定回傳 `#t` 或 `#f`

開場白的另外四條規則

- 不要多做假設。除非該題另外加了限制，否則資料就只有上一頁那些性質。
- 不必檢查輸入。假設傳進來的值一定屬於規格說的集合。
- 每個函式要有 **contract** 與 **usage** 註解，照本章的格式寫。
- 輔助函式可以自由定義，但每一個都要有自己的規格 (§1.3)。
- 課本給的例子不夠。先全部跑過，然後自己補其他例子。

第二條就是第 1 堂那個 `#f` 的另一面：

課本明文免除你檢查輸入的責任——代價是輸入不合格時，行為不保證。

三個硬性要求

一、每個函式都要有 contract 與 usage 註解。

```
;; swapper : Sym * Sym * S-list -> S-list  
;; usage: (swapper s1 s2 slist) returns slist with all  
;;         occurrences of s1 and s2 exchanged.
```

缺了直接扣分。課本 p.26 白紙黑字要求。

二、每個輔助函式要有自己的獨立規格。

No Mysterious Auxiliaries。1.33 一定會產生輔助函式。

三、自己補測試。

課本說「the given examples are not adequate」。

每題至少補兩個課本沒給的例子，包含邊界情況。

常見錯誤清單

錯誤	症狀
沒寫 contract 就開始寫	寫到一半不知道該回傳什麼
分支數目跟 production 對不上	漏 case，特定輸入爆掉
輔助函式只說明初始呼叫	自己也看不懂三天前寫的程式
用 <code>equal?</code> 比 symbol	語意不精確——symbol 要比的是同一性，課本比 symbol 一律用 <code>eqv?</code>
只測課本給的例子	邊界沒測到（空 list、 $n = 0$ 、找不到）

關於 1.31 和 1.33 的順序

1.31 要你寫 bintree 的介面：

```
(leaf 26)           ; 建構子
(interior-node 'red left right) ; 建構子
(leaf? t)           ; 判斷式
(lson t) (rson t)   ; 取出子樹
(contents-of t)     ; 取出內容
```

題目明說 **contents-of** 要同時能用在 leaf 和 interior node 上——

leaf 拿到的是它的值，interior node 拿到的是它的 symbol。漏掉這點，1.33 會做不出來。

1.33 必須用 1.31 的介面來寫，不可以直接拆 list。

為什麼？下週 Ch2.1 會給出完整的理由（representation independence）。

現在你只要照做——然後下週回來看你自己寫的程式碼。

AI 使用原則（重申）

歡迎：

課文看不懂，叫 AI 解釋。你卡在某個概念，跟 AI 辯論。

強烈不建議：

作業叫 AI 寫。

課堂演練

三題，現場做

演練的流程（每題都照這個走）

1. 寫 contract
2. 寫出輸入資料的 BNF
3. 依 BNF 畫出分支骨架（先不填內容）
4. 一格一格填
5. 到 REPL 測——包含邊界情況

不要跳過第 1、2 步。今天要練的是流程。

這三題都不在 HW1 裡。放心做。

演練 1：duple (Ex 1.15)

```
(duple 2 3)           ; => (3 3)
(duple 4 '(ha ha))    ; => ((ha ha) (ha ha) (ha ha) (ha ha))
(duple 0 '(blah))     ; => ()
```

先問自己：遞迴的對象是誰？

```
Nat ::= 0 | (successor Nat)
```

x 不需要被分解，它只是被複製。有歸納結構的只有 **n**。

演練 1：骨架

```
;; duple : Int * SchemeVal -> Listof(SchemeVal)
;; usage: if n >= 0, then (duple n x) = a list of n copies of x
(define duple
  (lambda (n x)
    (if (zero? n)
        ???
        ???)))
```

給你 2 分鐘。填完之後，問自己：為什麼這個遞迴保證會停？

演練 2 : list-set (Ex 1.19)

```
(list-set '(a b c d) 2 '(1 2))  
; => (a b (1 2) d)
```

這題兩個參數同時在遞迴。

```
List ::= () | (SchemeVal . List)  
Nat  ::= 0 | (successor Nat)
```

兩個歸納結構 $\Rightarrow$ 要想清楚 case 怎麼分。

演練 2：case 分析

lst	n	該做什麼
'()	任意	?
非空	0	?
非空	> 0	?

先把這張表填滿，再寫程式。

提示：這題跟課本 §1.2.2 的 `nth-element` 是同一個骨架。
差別只在「回傳什麼」。

演練 3：list-index (Ex 1.23) ★ 今天的重頭戲

```
(list-index number? '(a 2 (1 3) b 7))    ; => 1
(list-index symbol? '(a (b c) 17 foo))    ; => 0
(list-index symbol? '(1 2 (a b) 3))       ; => #f
```

先照配方試試看：

```
(define list-index
  (lambda (pred lst)
    (if (null? lst)
        #f
        ???)))
```

你會卡住。想清楚卡在哪裡。

演練 3：卡在哪

假設 `(list-index pred '(2 (1 3) b 7))` 回傳 `0`。

我要的答案是 `1`。

跟 `number-elements` 一模一樣的問題：

子結構不知道自己被切掉了幾個元素。

所以呢？

演練 3：用第 2 堂的招

```
;; list-index : Pred * List -> Int | #f
(define list-index
  (lambda (pred lst)
    (list-index-from pred lst 0)))

;; list-index-from : Pred * List * Int -> Int | #f
;; usage: returns the 0-based position, counting from n, of
;;        the first element of lst satisfying pred;
;;        #f if no element satisfies pred.
(define list-index-from
  (lambda (pred lst n)
    ???)))
```

注意 **usage** 註解：它說明了所有 `n` 的行為，不只 `n = 0`。

這就是 No Mysterious Auxiliaries。

三題的共通結構

題	遞迴對象	需要 context argument ?
1.15 <code>duple</code>	<code>n</code>	否
1.19 <code>list-set</code>	<code>lst</code> 與 <code>n</code>	否
1.23 <code>list-index</code>	<code>lst</code>	可以用（也可以不用）

判準：當「答案需要知道自己在原始結構中的位置」時，
你就需要一個 context argument。

下週預告

week 3 : Ch2.1-2.2 資料抽象

- interface 與 implementation 的分離
- 同一個介面、多種表示法
- environment 的第一個實作

本週請完成：

- [] HW1 開始動手 (week 4 交)

本日結論

資料的形狀決定程式的形狀。
當資料沒有形狀時，你就自己造一個。

第一句是 Ch1.2。

第二句是 Ch1.3。之後的章節會有類似的事——

Ch3 的 environment、Ch4 的 store，文法裡都沒有，都得自己造出來的。

問題時間

1. Email: laurence@replware.dev
2. Office Hour: 週四下午 1:00~2:00，請先跟我約